

Homework 2 - Natural language inference (NLI)

Aur Marina Iuliana, 1809715

Multilingual Natural Language Processing (MNLP) course
A.Y. 2023-2024

1 Introduction

Natural language inference (NLI) is a NLP classification task designed to detect the relationship between a premise and a hypothesis. Specifically, given an hypothesis h , the aim of the task is to analyze its content and determine whether a hypothesis is true (*entailment*), false (*contradiction*), or undetermined (*neutral*) in relation to a certain premise p .

2 Provided Datasets

2.1 FEVER Dataset

For this task, a randomly 25% down-sampled subset of the original **FEVER** (Fact Extraction and VERification) dataset was employed. Additionally, the dataset is also augmented with semantic annotations. The schema of the dataset is illustrated in 1. Additionally, an example of one sample in the dataset is provided in 1 to enhance comprehension.

2.2 Adversarial FEVER NLI Dataset

Adversarial FEVER NLI Dataset includes new samples generated manually and using GPT-4o model applied to the FEVER dataset. The aim of this dataset is to assess the generalization capability of models in NLI task when exposed to more complex "*perturbed*" data.

3 Model

DeBERTa V3 base model from HuggingFace library is an advanced pre-trained Large Language Model (LLM), which improves the BERT and RoBERTa models using disentangled attention mechanism and enhanced mask decoder. The model is chosen for the task due to its capability to handle **long text sequences** effectively.

4 Exploratory data analysis (EDA)

An exploratory data analysis (EDA) was conducted to gain insights into the data. Firstly, **label fre-**

quency investigated to identify potential class inconsistencies in the task. While the test and validation datasets demonstrated balance in the label frequency, the training dataset exhibits a significant dominance of class 0 (entailment) over class 1 (contradiction) and class 2 (neutral), underscoring the need for data augmentation [1]. Figure 1 offers insights into **distribution of sentence lengths** in the training set, showing that the majority of premise sentences and hypothesis contain respectively fewer than 400 and 60 characters. (*Note: the same figures for test and validation sets can be found in the notebook*).

5 Data Augmentation

To enhance the model's **robustness to minor variations** in the original dataset and **mitigate class imbalance**, data augmentation technique was employed. Based on EDA analysis, it was noted an imbalance of class 1 (contradiction) and class 2 (neutral) frequency. Consequently, it was decided to augment the data for these classes. **DeBERTa V3 base pre-trained on NLI task** from HuggingFace library is used to **check the quality** of our results and avoid "*polluting*" the model with false positives.

5.1 Neutral Adversarial Examples

These adversarial examples are generated using both examples from class 0 (entailment) and class 1 (contradiction). For each approach, a unique seed was chosen to ensure reproducibility and avoid recurring phrase selection. After generating the adversarial examples, the model is employed to predict their class: if the predicted class matches the desired class, then these examples are stored in a JSONL file for subsequent addition in the original dataset.

5.1.1 First Approach: Swap hypothesis

This approach is based on the idea that of swapping the current entailment/contradiction hypothesis with the next one and pairing the last one with the first, introducing an undetermined relationship between premise and hypothesis. This approach is very simple but may introduce ambiguity in the model since the hypothesis lose the context of the premise. For this approach are provided as input 5,000 examples from class 0 (entailment) and 5,000 examples from class 1 (contradiction), but only **4,027 examples** passed the quality check, highlighting the limitations of this approach. The execution time required to generate the adversarial examples in this approach is **33.572 minutes**.

5.1.2 Second Approach: Generate Neutral Text with GPT2

In this approach, **GPT-2** model from HuggingFace library is utilized to generate neutral hypothesis from a given prompt. For this approach are provided as input 250 examples from class 0 (entailment) and 250 examples from class 1 (contradiction) since it requires more computation time, but only **312 examples** passed the quality check. The execution time required to generate the adversarial examples in this approach is **18.371 minutes**.

5.2 Contradiction Adversarial Examples

These adversarial examples are generated using examples from class 0 (entailment). For each approach, a unique seed was chosen to ensure reproducibility and avoid recurring phrase selection.

5.2.1 First Approach: Introduce negation in the hypothesis

In this approach, we randomly select one negation sentence from the list (referenced in 2) and prepend it to the hypothesis to transform it into a contradiction. For this approach are provided as input 1,000 examples from class 0 (entailment), but only **715 examples** passed the quality check. The execution time required to generate the adversarial examples in this approach is **3.609 minutes**, making it highly efficient.

5.2.2 Second Approach: Swap Agent-Patient using Semantic Role Labeling (SRL)

In this approach, **English Propbank corpus** is employed to identify agents (ARG0) and patients (ARG1) and we swapped them to introduce a contradiction. For this approach are provided as input

1,000 examples from class 0 (entailment), but only **353 examples** passed the quality check, highlighting the limitations of this approach and the need to introduce further assumptions. The execution time required to generate the adversarial examples in this approach is **5.152 minutes**.

5.2.3 Third Approach: Numerical Inference

In this approach, when encountering a premise containing a date, we increment the date in the hypothesis by 1 ('in' + (date + 1)), thereby introducing a contradiction. This approach is simple to implement, but its effectiveness is limited to specific cases. For this approach are provided as input 10,000 examples from class 0 (entailment), but only **2835 examples** passed the quality check, highlighting the limitations of this approach and the need to introduce further assumptions. The execution time required to generate the adversarial examples in this approach is **23.533 minutes**.

6 Performance Evaluation

6.1 Performance Evaluation using original data

A summary of the models performance using original data can be observed 3 and 4. Figure 2 includes the model confusion matrices in FEVER Test-Set and Adversarial Test-Set using original data. Analyzing the results, we noticed that the model achieved an **accuracy of 0.69** in FEVER dataset and **0.50** in Adversarial dataset.

6.2 Performance Evaluation using augmented data

A summary of the models performance using augmented data can be observed 5 and 6. Figure 3 includes the model confusion matrices in FEVER Test-Set and Adversarial Test-Set using augmented data. Analyzing the results, we noticed a slight improvement in the model performance, which achieved an **accuracy of 0.73** in FEVER dataset and **0.54** in Adversarial dataset, highlighting the need for further considerations and approaches.

7 Code Instructions

The script was run on Google Colab, while the dataset and the adversarial examples files were uploaded on Google Drive to make them accessible in the notebook. GPU is necessary to run models and avoid execution time problems.

8 Tables and Figures

Listing 1: FEVER Dataset schema

```
{
  "id": ..., # the FEVER dataset ID
  "premise": ..., # the context in FEVER
  "hypothesis": ..., # the query in FEVER
  "label": ..., # mapped version of FEVER
}
```

Listing 2: Negations for Contradiction Adversarial Examples

```
negations = ['It is not the case that ', 'Not necessary ',
             'It is not true that ', 'In contrast to ']
```

Listing 3: Performance Evaluation with original data in FEVER Test-Set

```
{ 'eval_loss ': 0.8351636528968811,
  'eval_accuracy ': 0.6974202011368605,
  'eval_f1 ': 0.6806872813013493,
  'eval_precision ': 0.6913076727662629,
  'eval_recall ': 0.6974202011368605,
  'eval_runtime ': 29.89,
  'eval_samples_per_second ': 76.514,
  'eval_steps_per_second ': 9.568,
  'epoch ': 1.0}
```

Listing 4: Performance Evaluation with original data in FEVER Test-Set

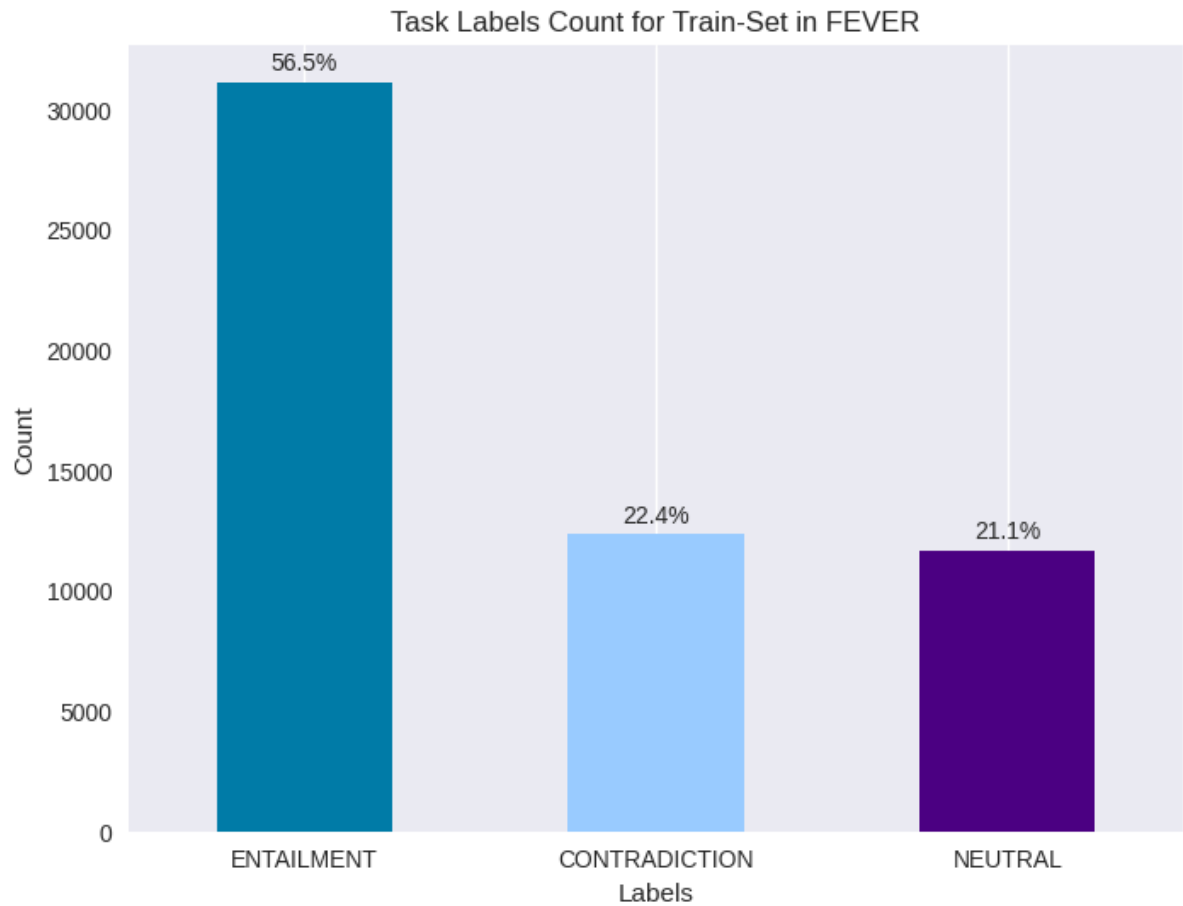
```
{ 'eval_loss ': 1.3667525053024292,
  'eval_accuracy ': 0.5074183976261127,
  'eval_f1 ': 0.5020659035563935,
  'eval_precision ': 0.5402597019143368,
  'eval_recall ': 0.5074183976261127,
  'eval_runtime ': 5.9238,
  'eval_samples_per_second ': 56.889,
  'eval_steps_per_second ': 7.259,
  'epoch ': 1.0}
```

Listing 5: Performance Evaluation with augmented data in FEVER Test-Set

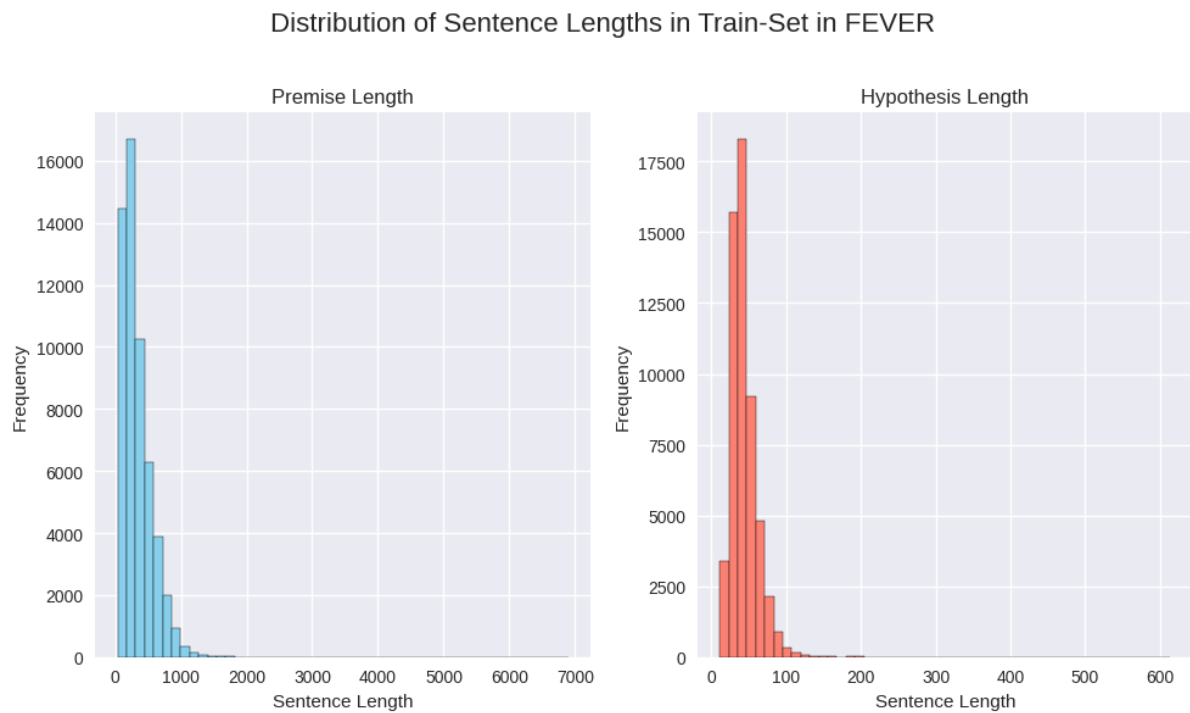
```
{ 'eval_loss ': 0.7648345828056335,
  'eval_accuracy ': 0.7319632706602536,
  'eval_f1 ': 0.719401040126269,
  'eval_precision ': 0.7273607830167446,
  'eval_recall ': 0.7319632706602536,
  'eval_runtime ': 30.0508,
  'eval_samples_per_second ': 76.105,
  'eval_steps_per_second ': 9.517,
  'epoch ': 1.0}
```

Listing 6: Performance Evaluation with augmented data in FEVER Test-Set

```
{ 'eval_loss ': 1.3162816762924194,  
  'eval_accuracy ': 0.543026706231454,  
  'eval_f1 ': 0.5438815853727647,  
  'eval_precision ': 0.5454693620741705,  
  'eval_recall ': 0.543026706231454,  
  'eval_runtime ': 4.9268,  
  'eval_samples_per_second ': 68.402,  
  'eval_steps_per_second ': 8.728,  
  'epoch ': 1.0}
```

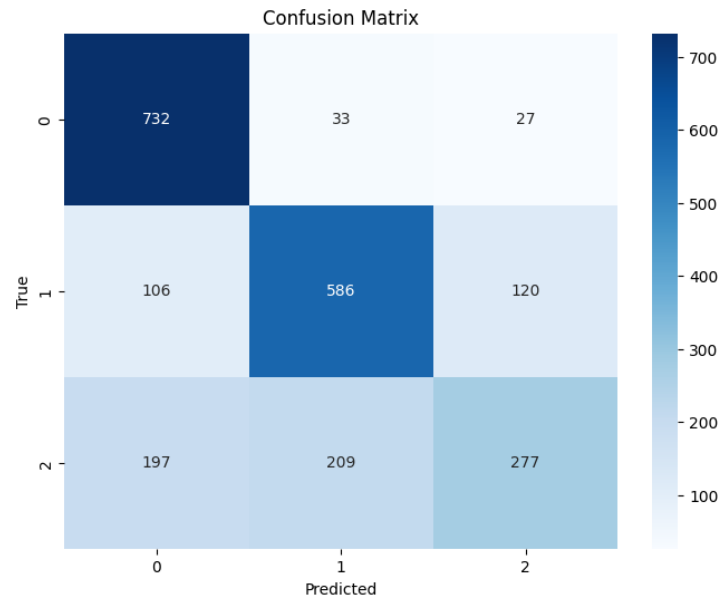


(a) Label counts for Train-Set in FEVER

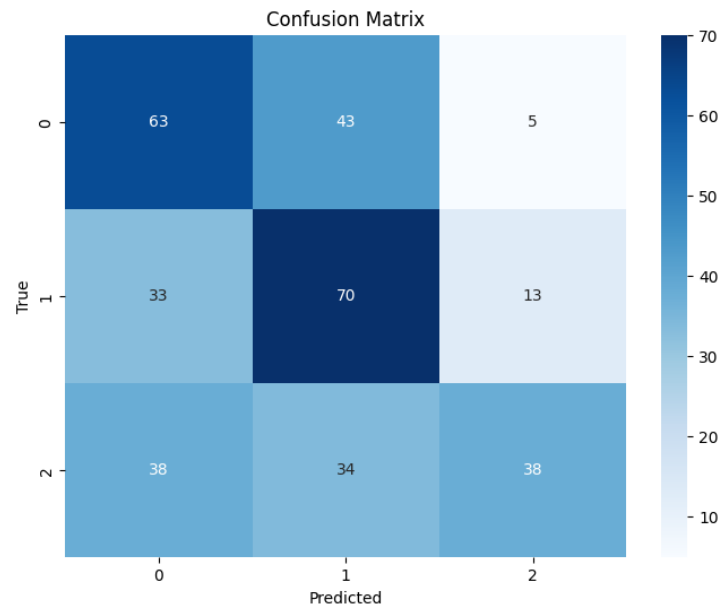


(b) Distribution of sentence lengths for Train-Set in FEVER

Figure 1: Data insights

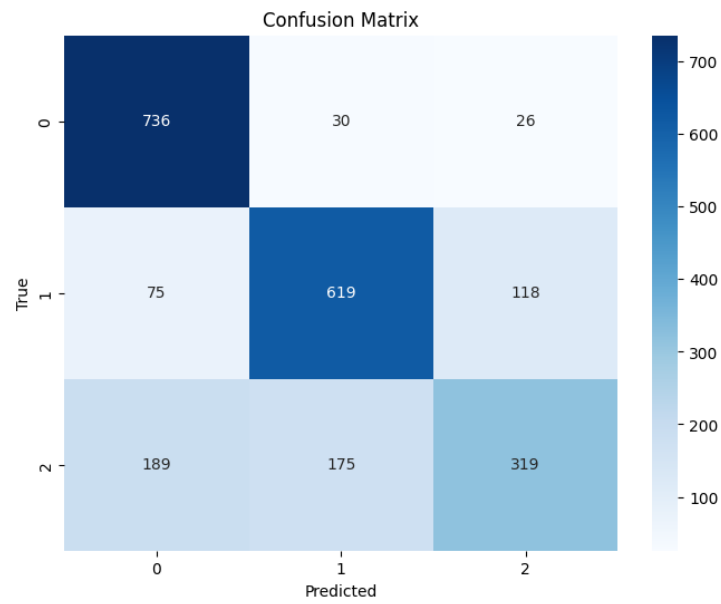


(a) DeBERTa V3 model confusion matrix using FEVER Test-Set

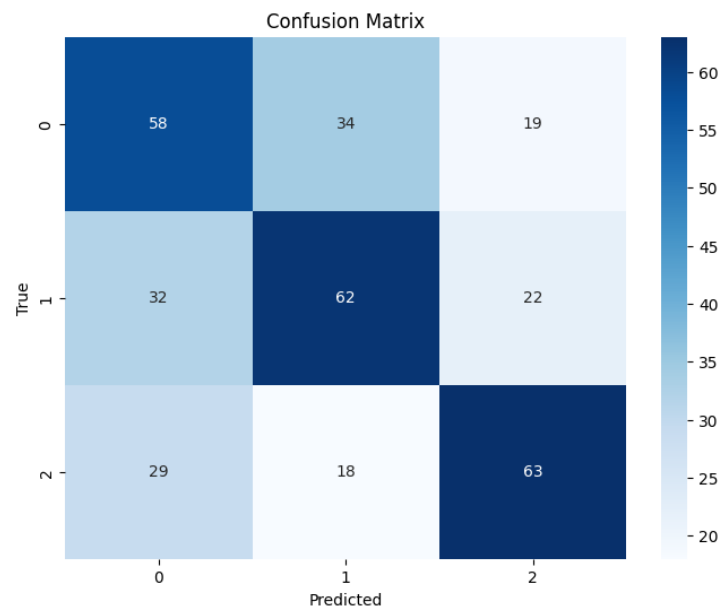


(b) DeBERTa V3 model confusion matrix using Adversarial Test-Set

Figure 2: Performance Evaluation using Original Data



(a) DeBERTa V3 model confusion matrix using FEVER Test-Set



(b) DeBERTa V3 model confusion matrix using Adversarial Test-Set

Figure 3: Performance Evaluation using Augmented Data